Java version companion for the senior engineer interview

9 MAIN SECTIONS • PRINT-READY EDITION

In this guide

1. The release map to memorize 2
2. Java 8: the foundation 3
3. Java 9: modules, REPL, and collection factories 4
4. Java 11 (LTS): HTTP and convenient syntax 6
5. Features available in Java 17 (LTS) 7
6. Java 21 (LTS): concurrency, patterns, and ordered collections 9
7. Java 25 (LTS): scoped context and smaller objects 10
8. Java 26: preview patterns, HTTP/3 client, and startup work 12
9. Five answers to rehearse before the interview 13

Prepared for Sreedhar | October 22, 2026 | Use alongside interview-plan.md and interview-study-guide.md

This guide explains the features you listed in plain English and gives you short, professional answers to say aloud. It covers the selected releases through Java 26. A feature's first preview, first final release, and the LTS release in which teams commonly adopt it can be different. Say which one you mean. The version used by your interview team is unknown, so ask before writing version-specific code.

Interview priority: Spend most of your time on Java 8, the features available by Java 17, and Java 21 virtual threads. Know the Java 25/26 features at a high level, especially which remain preview. This is a companion to your coding preparation, not a replacement for it.

1. The release map to memorize

Feature	First final release or current status	Short memory cue
Lambdas, streams, Optional, default/static interface methods, `java.time`	Java 8	Functional programming and modern time
Module system, JShell, `List.of` / `Set.of` / `Map.of`	Java 9	Modules, shell, collection factories
Local variable `var`	Java 10	Local variables only; **not** the Java 11 lambda addition
Standard `HttpClient`, `var` lambda parameters, `java File.java`	Java 11 (LTS)	HTTP, lambda syntax, single-file launch
Switch expressions	Java 14	Switch returns a value
Text blocks	Java 15	Multiline strings
Records and `instanceof` patterns	Java 16	Data carriers and binding variables
Sealed classes; strong encapsulation of JDK internals by default	Java 17 (LTS)	Controlled subtypes and migration rules
Virtual threads, record patterns, switch patterns, sequenced collections, Generational ZGC	Java 21 (LTS)	Lightweight I/O concurrency and richer patterns
Scoped values, flexible constructor bodies, module imports; compact object headers as a product feature	Java 25 (LTS)	Scoped context and refined language/runtime
Structured concurrency	**Preview** in Java 25 (fifth) and Java 26 (sixth)	Related subtasks as one unit; API can still change
Primitive patterns	**Fourth preview** in Java 26	Patterns can include primitives
HTTP/3 in the built-in HTTP client; AOT object caching with any GC	Java 26 (non-LTS)	Client protocol and startup cache

Oracle identifies Java 8, 11, 17, 21, and 25 as LTS releases; Java 9 and 26 are non-LTS. This is a support designation, not a statement that every feature was invented in that LTS release. Sources: Oracle support roadmap, Oracle language release table, JDK 21 release notes, JDK 25 release notes, and JDK 26 release notes.

The four corrections most likely to impress an interviewer

1. **Virtual threads were finalized in Java 21**, not introduced in Java 25. Java 25 continues to improve the ecosystem around them. JEP 444.

2. **"Java 17 features" is a convenient study grouping.** Switch expressions became final in 14, text blocks in 15, records and `instanceof` patterns in 16, and sealed classes in 17. All are available when running Java 17. Oracle language changes.
3. **Java 25 did not finalize structured concurrency.** Scoped values are final in 25; structured concurrency remains preview in 25 and 26. JDK 25, JDK 26.
4. **Generational ZGC's main distinction is generational collection.** ZGC was already designed for low pauses. The generational change targets better collection efficiency, allocation behavior, and memory overhead while retaining low-latency goals; do not promise a specific pause reduction without measurements. JDK 21 notes.

2. Java 8: the foundation

Default and static interface methods

Plain English: An interface can include a `default` method with an implementation. This lets a library add useful behavior while existing implementers keep working in many common cases. A `static` interface method belongs to the interface itself.

JAVA

```java
interface PaymentValidator {
    boolean valid(String id);

    default boolean validAndPresent(String id) {
        return id != null && valid(id);
    }

    static boolean hasText(String id) {
        return id != null && !id.isEmpty();
    }
}
// Call the static method as PaymentValidator.hasText(id).
```

Say it professionally: "Default methods let an interface evolve while preserving many existing implementations. Static interface methods group related utilities with the contract. If two inherited defaults conflict, the implementing class must resolve the conflict explicitly."

Watch for: Adding a default method does not make every conceivable source or behavior conflict disappear. A class method takes precedence over an interface default method. Java 8 language enhancements.

Lambdas, streams, and Optional

Plain English: A lambda is a small function passed as a value. A stream describes a pipeline of operations on data. `Optional<T>` represents a result that may be absent.

JAVA

```
List<String> approved = payments.stream()
        .filter(p -> p.isApproved())
        .map(p -> p.reference())
        .collect(Collectors.toList());
Optional<Payment> found = repository.findById(id);
```

Say it professionally: "Lambdas and streams make data transformations concise. Intermediate stream operations are lazy; a terminal operation executes the pipeline. I use Optional mainly to express a possibly missing return value and handle the empty case explicitly."

Watch for: `map` transforms each item; `flatMap` flattens nested results. A stream is normally consumed once. Avoid side effects and shared mutable state in stream pipelines. `orElse` evaluates its fallback immediately; `orElseGet` defers it. Parallel streams are not automatically faster. Optional is not a blanket replacement for every nullable field. JDK 8 overview.

New date and time API: `java.time`

Plain English: `LocalDate` is a calendar date. `LocalDateTime` has date and clock time but no time zone. `Instant` is a point on the UTC timeline. `ZonedDateTime` adds a named zone and its rules. These types make time handling clearer than many legacy date APIs.

JAVA

```
Instant createdAt = Instant.now();
LocalDate dueDate = LocalDate.parse("2026-10-23");
ZonedDateTime meeting = dueDate.atTime(18, 0)
        .atZone(ZoneId.of("America/Phoenix"));
```

Say it professionally: "I use `Instant` for timestamps and a named `ZoneId` for user-facing schedules. A `LocalDateTime` alone cannot identify an absolute moment. The `java.time` types are immutable, which also simplifies shared use."

Watch for: `java.util.Date` still exists; `java.time` did not remove it. A fixed UTC offset is not always a substitute for a named zone, because many zones change offset seasonally. Test daylight-saving boundaries where relevant. JDK 8 date-time overview.

3. Java 9: modules, REPL, and collection factories

Java Platform Module System

Plain English: A module names a group of packages and states what it requires and exports. A package that is not exported is hidden from ordinary access outside the module. This helps make application boundaries explicit.

JAVA

```
module com.example.payments {
    requires java.sql;
    exports com.example.payments.api;
}
```

Say it professionally: "The module system adds explicit dependencies and exported packages through `module-info.java`. It helps with encapsulation and custom runtime images. An existing classpath application can still run without being rewritten as named modules."

Watch for: Java 9 introduced the module system, but **strong encapsulation of JDK internals by default** was strengthened in Java 17. This matters when upgrading older libraries that use reflection on internal JDK classes. A migration flag such as `--add-opens` can help temporarily, but a supported API or updated library is the better long-term fix. JDK 17 migration notes.

JShell

Plain English: JShell is an interactive Java console. Type an expression or declaration and see the result immediately. It is useful for learning and small checks.

EXAMPLE

```
jshell> 2 + 3
$1 ==> 5
```

Say it professionally: "JShell is the JDK's REPL. I use it to check small Java expressions and APIs quickly. It does not replace a reproducible project build and test suite." JDK 9 release notes.

`List.of`, `Set.of`, and `Map.of`

Plain English: These factory methods create compact, unmodifiable collections.

JAVA

```
List<String> statuses = List.of("PENDING", "SUCCESS");
Set<String> roles = Set.of("ADMIN", "VIEWER");
Map<String, Integer> limits = Map.of("USD", 100, "EUR", 90);
```

Say it professionally: "The Java 9 collection factories are useful for small fixed collections. They reject nulls, and `Set.of` rejects duplicate elements. Unmodifiable means the collection cannot be structurally changed; it does not deeply freeze mutable objects stored inside it."

Watch for: `List.of` may contain duplicate values, but `Set.of` cannot. `Map.of` rejects duplicate keys. Do not rely on the iteration order of `Set.of` or `Map.of`. Java 9 collection factories.

4. Java 11 (LTS): HTTP and convenient syntax

Standard HTTP Client API

Plain English: `java.net.http.HttpClient` is a built-in client for calling HTTP services. It supports HTTP/1.1 and HTTP/2, and both synchronous and asynchronous calls. It became standard in Java 11 after incubation.

JAVA

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder(URI.create("https://example.com"))
        .GET().build();
CompletableFuture<HttpResponse<String>> pending = client.sendAsync(
        request, HttpResponse.BodyHandlers.ofString());
```

Say it professionally: "Java 11 standardized a built-in HTTP client with asynchronous calls that return `CompletableFuture`. For production use I still configure timeouts, handle unsuccessful responses, and control concurrency. HTTP/3 support was added later, in Java 26." JDK 11 release notes, JDK 26 release notes.

`var` in lambda parameters

Plain English: Java 10 introduced `var` for local variables. Java 11 added the option to use `var` for lambda parameters too.

JAVA

```
BiFunction<String, String, String> join = (var left, var right) -> left + right;
```

Say it professionally: "The Java 11 change allows `var` in implicitly typed lambda parameter lists. It is especially useful when a parameter needs an annotation. If I use `var`, I apply it consistently to all parameters in that lambda."

Watch for: `var` is type inference, not dynamic typing. `(var a, String b) -> ...` is invalid. `var` is not used for method return types or fields. Oracle language changes.

Single-file source execution

Plain English: For a simple program, `java Hello.java` compiles it in memory and runs it. You do not have to call `javac` first.

EXAMPLE

```
java Hello.java
```

Say it professionally: "Single-file source execution is convenient for small tools, demonstrations, and learning. I still use a normal build for applications with several sources, dependencies, and tests."

Watch for: It still compiles the source; it simply combines compilation and execution for you. JDK 11 release overview.

5. Features available in Java 17 (LTS)

The items below are useful to a Java 17 developer. Their **first final releases differ**: switch expressions 14, text blocks 15, records and `instanceof` patterns 16, sealed classes

1. Oracle language changes.

Switch expressions: final in Java 14

Plain English: A switch can produce a value. Arrow cases do not fall through by default, which removes a common source of bugs.

JAVA

```
String label = switch (status) {
    case APPROVED -> "Ready";
    case REJECTED -> "Declined";
    default -> "Review";
};
```

Say it professionally: "A switch expression returns a value, which is useful for a complete mapping from input cases to an output. Arrow cases avoid accidental fall-through. I handle all possible inputs explicitly or use an appropriate default."

Text blocks: final in Java 15

Plain English: A text block uses triple quotes for a multiline string. It reduces escape characters in SQL, JSON, and test data.

JAVA

```
String query = """
        SELECT account_id, SUM(amount)
        FROM payments
        GROUP BY account_id
        """;
```

Say it professionally: "Text blocks improve readability for multiline literals. They do not protect against SQL injection; external values still need parameter binding."

Records: final in Java 16

Plain English: A record is a concise data carrier. The compiler supplies a constructor, accessors, equality, hash code, and a string representation based on its components.

JAVA

```
record PaymentKey(String tenantId, String requestId) {
}
```

Say it professionally: "I use records for clear value-oriented data such as request keys or result DTOs. A record's fields cannot be reassigned, but it is only shallowly immutable if a component refers to a mutable object."

Watch for: A record is not a JPA entity replacement in every framework setup. Validate invariants in a compact constructor when necessary; avoid exposing mutable components without defensive copies.

Pattern matching for `instanceof`: final in Java 16

Plain English: The type test also creates a variable of that type, so an explicit cast is unnecessary.

JAVA

```
if(value instanceof
String name){
        System.out.

println(name.length());
        }
```

Say it professionally: "An `instanceof` pattern combines type checking and safe variable binding. It simplifies code that has to inspect an object of a broader type."

Sealed classes: final in Java 17

Plain English: A sealed class or interface lists which classes may directly implement or extend it. This is useful for a known set of outcomes.

JAVA

```
sealed interface PaymentResult permits Approved, Declined {
}

record Approved(String reference) implements PaymentResult {
}

record Declined(String reason) implements PaymentResult {
}
```

Say it professionally: "Sealed types make a closed domain model explicit. They help readers and the compiler reason about the allowed alternatives. I use them when the set of direct subtypes is intentionally controlled."

Watch for: A permitted subclass must follow the required inheritance rules: final, sealed, or non-sealed, with records already final. Do not make an extensible plugin contract sealed by accident.

6. Java 21 (LTS): concurrency, patterns, and ordered collections

Virtual threads: final in Java 21

Plain English: A virtual thread is a lightweight Java thread. Many tasks that spend most of their time waiting for I/O can be written in a straightforward blocking style without holding one operating-system thread per waiting task.

JAVA

```
try(var executor = Executors.newVirtualThreadPerTaskExecutor()){
Future<String> result = executor.submit(() -> fetchAccount());
    System.out.

println(result.get());
        }
```

Say it professionally: "Virtual threads improve scalability for workloads with many concurrent I/O waits. They do not make CPU work faster or remove limits on database connections. I create a virtual thread per task and put limits around scarce downstream resources."

Watch for: Do not pool virtual threads. Blocking code that holds a scarce connection is still bounded by that connection pool. Pinning details changed after Java 21, so qualify any version-specific claim. JEP 444.

Record patterns and pattern matching for switch: final in Java 21

Plain English: A record pattern unpacks a record into its components. A pattern switch chooses behavior by type and can bind values in each case.

JAVA

```
record Amount(String currency, long cents) {
}

String describe(Object value) {
    return switch (value) {
        case Amount(String currency, long cents) -> currency + " " + cents;
        case String text -> text;
        case null -> "missing";
        default -> "other";
    };
}
```

Say it professionally: "Record patterns remove manual casts and accessor calls when deconstructing records. Pattern switches can be exhaustive when the input type has a closed set of alternatives, such as a sealed hierarchy."

Watch for: `case null` is explicit; it is not automatically handled by `default` in the way many developers assume. Case ordering matters when a broad pattern would dominate a narrower one. Oracle language changes.

Sequenced collections

Plain English: The JDK added common interfaces for collections with a defined encounter order. They provide a shared way to ask for the first or last element and to view the order in reverse.

JAVA

```
SequencedCollection<String> events = new ArrayList<>();
events.

add("received");
events.

add("completed");

String first = events.getFirst();
```

Say it professionally: "Sequenced collections give ordered lists, sets, and maps more consistent APIs for first, last, and reversed views. I still check the specific implementation's ordering and mutation rules."

Watch for: Not every `Collection` is sequenced. A `HashSet` does not suddenly gain a meaningful stable order. JEP 431.

Generational ZGC

Plain English: Many objects die young. Generational ZGC treats young and old objects separately so it can collect young garbage more efficiently.

Say it professionally: "Generational ZGC extends a low-latency collector with young and old generations. It can reduce allocation stalls, memory overhead, and GC CPU use for suitable workloads. I would compare pause time, throughput, memory use, and allocation pressure under our actual load before choosing a collector."

Watch for: The Java 21 feature was not a universal guarantee of dramatically shorter pauses. GC availability, defaults, and options can change across JDK releases. JDK 21 release notes.

7. Java 25 (LTS): scoped context and smaller objects

Scoped values: final in Java 25

Plain English: A scoped value lets a caller bind a value for a limited call scope. Code inside that scope can read it, including appropriate child tasks. It suits context that should flow down the call tree without being changed.

JAVA

```
static final ScopedValue<String> REQUEST_ID = ScopedValue.newInstance();

ScopedValue.

where(REQUEST_ID, "req-123").

run(() ->{

audit(REQUEST_ID.get());
        });
```

Say it professionally: "Scoped values let me share immutable request context through a bounded call scope. They are easier to reason about than mutable thread-local context, particularly with virtual threads. I still pass ordinary method parameters when that makes the data dependency clearer."

Watch for: The reference is scoped; the object you bind is not magically deeply immutable. Understand how context propagates to child tasks in the API you actually use. JEP 506.

Structured concurrency: fifth preview in Java 25

Plain English: When one request starts related subtasks, structured concurrency treats them as a group. The parent can wait for them, handle failure, and cancel sibling work as one bounded operation.

Say it professionally: "Structured concurrency is promising for request workflows that fork several related calls and must join them before returning. In Java 25 it is still a preview API, so I would avoid presenting its current syntax as permanent and check the exact JDK version before adopting it."

Watch for: Java 26 carries a **sixth preview**. Preview features require enabling preview at compile and run time, and the API can change. Virtual threads and structured concurrency are related but distinct: virtual threads are a thread implementation, while structured concurrency organizes task lifetime. JDK 25 notes, JDK 26 notes.

Flexible constructor bodies

Plain English: A constructor can do some safe work before calling `super(...)` or `this(...)`. This helps validate input before the parent constructor runs.

JAVA

```java
class PaymentLimit extends BaseLimit {
    PaymentLimit(int value) {
        if (value < 0) throw new IllegalArgumentException("negative limit");
        super(value);
    }
}
```

Say it professionally: "Flexible constructor bodies allow validation and other safe setup before an explicit constructor invocation. The early statements cannot use the object under construction, so they cannot call instance methods on `this` before the superclass is initialized." JDK 25 notes.

Compact object headers

Plain English: Each Java object has metadata in addition to its fields. Compact object headers can reduce that metadata and therefore reduce heap use in object-heavy applications.

Say it professionally: "Compact object headers became a product feature in Java 25. The potential benefit is a smaller heap footprint and possibly better performance for some workloads, but I would benchmark our object mix. In Java 25 the option is available but disabled by default." JDK 25 release notes.

Module import declarations

Plain English: One declaration can import the public packages exported by a module. It is a convenience for source code, especially small programs.

```java
import module java.base;
```

Say it professionally: "A module import declaration brings exported packages of a module into scope without listing each package. It does not grant access to non-exported internal packages, and it does not replace a module's own `requires` declaration." JEP 511.

8. Java 26: preview patterns, HTTP/3 client, and startup work

Java 26 is **non-LTS**. These topics are useful to recognize, but they are less likely to matter in a coding round on an older LTS runtime. Oracle roadmap.

Primitive types in patterns: fourth preview

Plain English: Pattern matching is being extended so primitive types, such as `int` and `double`, can participate more consistently in `instanceof` and `switch`. Java 26 offers the fourth preview, not a final language feature.

Say it professionally: "Java 26 previews primitive patterns. They make pattern matching more uniform across reference and primitive values, while preserving rules around safe conversions. I would confirm the preview syntax and enablement for the exact JDK before using it in project code." Oracle language changes.

HTTP/3 in the built-in client

Plain English: Java 26's `HttpClient` can speak HTTP/3 to a supporting server. HTTP/3 uses QUIC over UDP rather than the TCP transport used by HTTP/1.1 and HTTP/2.

Say it professionally: "Java 26 adds HTTP/3 support to the standard HTTP client. A client or request opts in; HTTP/3 is not the default protocol. I would verify server support and measure connection behavior before choosing it for a service integration."

Watch for: This does **not** mean the JDK now supplies a general QUIC API or an HTTP/3 server. Protocol negotiation and fallback are details to confirm with the chosen configuration. JDK 26 release notes, JEP 517.

AOT object caching with any GC

Plain English: Ahead-of-time (AOT) caching saves some application setup work from an earlier run, so later starts can load it. Java 26 extends object caching to work with any HotSpot garbage collector, including ZGC.

Say it professionally: "This is a startup and warmup optimization from Project Leyden. Java 26 makes the AOT object cache compatible with all collectors, including ZGC. I would measure cold start and warmup for our application; it is not the same as compiling the whole application to a native executable."

Watch for: The feature extends AOT work introduced in earlier releases. It does not automatically accelerate every run without using the appropriate cache workflow. JDK 26 release notes.

9. Five answers to rehearse before the interview

"What changed from Java 8 to Java 17?" "Java 8 brought lambdas, streams, Optional, default interface methods, and the modern time API. Java 9 added modules and collection factories; Java 11 standardized the HTTP client. By Java 17, teams also had switch expressions, text blocks, records, `instanceof` patterns, and sealed classes. I would focus an upgrade on compatibility and testing, especially libraries that depend on JDK internals."

"What changed from Java 17 to Java 21?" "The biggest backend feature for me is virtual threads: they let a request-per-thread style scale better when requests wait on I/O. Java 21 also finalized record and switch patterns, added sequenced collections, and introduced Generational ZGC. I would still measure database pool limits and actual latency before claiming a performance benefit."

"Are virtual threads a Java 25 feature?" "They were finalized in Java 21. Java 25 finalized scoped values and kept structured concurrency in preview, which complement virtual-thread-based designs."

"Is a record immutable?" "Its component fields cannot be reassigned, and the compiler gives it value-oriented methods. It is shallowly immutable: a mutable object stored in a component can still change unless I copy or protect it."

"Should we adopt Java 26 immediately?" "I would first confirm our supported runtime, libraries, deployment platform, and reason for upgrading. Java 26 brings features such as HTTP/3 client support, while primitive patterns and structured concurrency are still preview. For production I would prioritize a supported version and validated performance over adopting a feature just because it is new."

Two-minute final recall

8: lambdas/streams/Optional, defaults, time. **9:** modules, JShell, `of`. **11:** HTTP client, lambda `var`, run one source file. **14–17:** switch, text blocks, records and `instanceof`, sealed. **21:** virtual threads, record/switch patterns, sequenced collections, Generational ZGC. **25:** scoped values final; structured concurrency preview; flexible constructors, compact headers, module imports. **26:** primitive patterns preview; HTTP/3 client and broader AOT cache support.

When asked about a feature you have not used, say: "I know what it provides, but I have not used it in production. I would first check our JDK version and its final or preview status, then validate the behavior with a small test."